

I²C Master Core

Version 1.00.00

2002-12-14

Organization

Konkuk University

1 Hwayang-dong, Gwangjin-gu, Seoul, 143-701, President: Yong-Beom Cho

South Korea

Phone: +82-02-456-3491

<http://vlsi.konkuk.ac.kr>

E-mail: ybcho@konkuk.ac.kr

Description/Abstract

I²C is a two-wire, bi-directional serial bus that provides a simple and efficient method of data exchange between devices. It is most suitable for applications requiring occasional communication over a short distance between many devices.

I²C defines 3 transmission speeds:

- Normal: 100Kbps
- Fast: 400Kbps
- High Speed: 3.5Mbps

Only normal and fast modes are supported directly.

The I²C standard is a true multi-master bus. This core, however, supports only single master operations, in which the core is the master.

Contents

1. Features and characteristics	2
2. Verification of features and characteristics	2
3. Block diagram.....	5
4. Register description	8
5. Timing diagram.....	10
6. Clock distribution.....	11
7. Bus interface & I/O configurations.....	12
8. Test description	15
9. Integration requirements	16
10. Version history	17
11. Known bugs	17
12. Application note.....	17
13. Deliverable list / format	17

1. Features and characteristics

1.1. General Information

1.1.1. IP type

- Soft IP

1.1.2. Basic function

- Only normal (100Kbps) and fast (400Kbps) modes are supported
- Single master operation
- Start/Stop/Repeated Start/Acknowledge generation
- Supports Clock Stretching/Wait state generation
- Software programmable clock frequency
- Software programmable acknowledge bit
- Fully synthesizable

1.1.3. Applications

- General-purpose processor peripheral as I²C-bus master core

1.2. Features

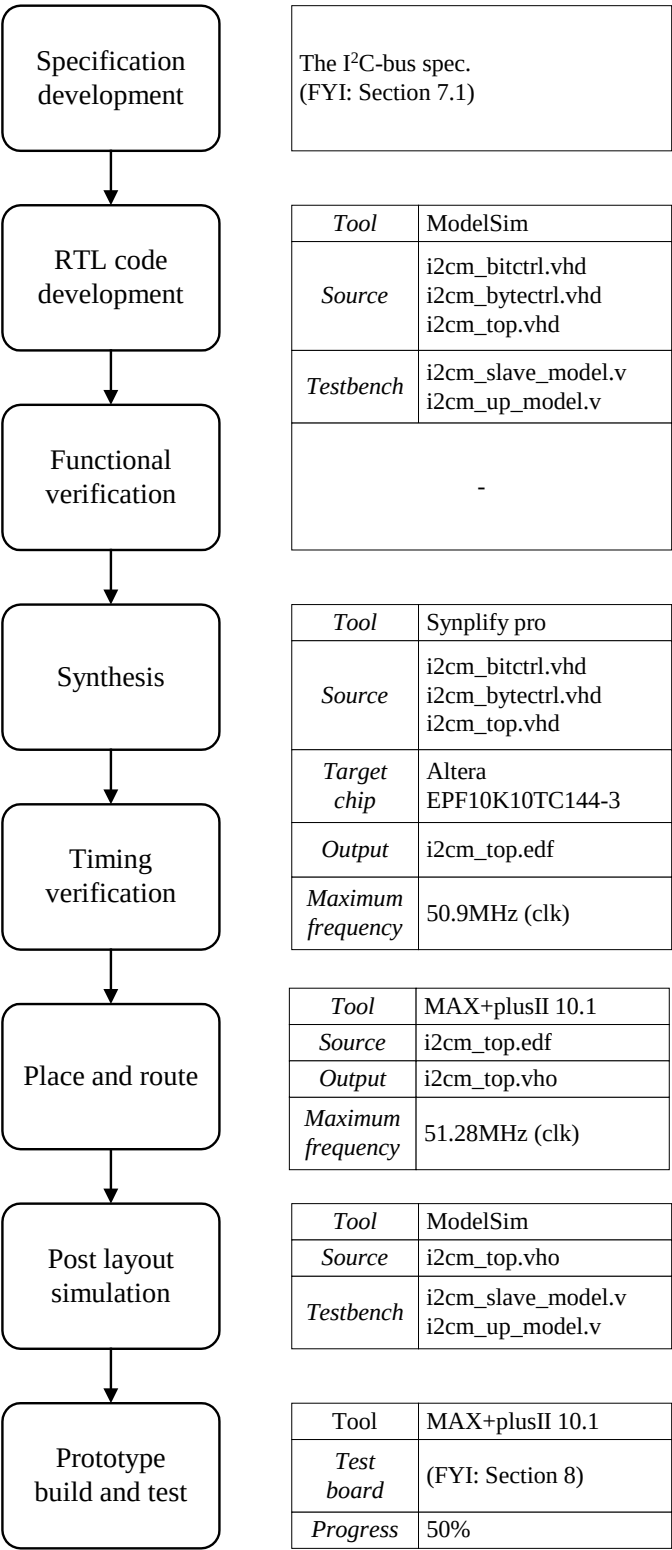
Supported mode	Normal and fast modes
IP size	The number of logic cells is 293. (Altera EPF10K10TC144-3)

2. Verification of features and characteristics

2.1. Reference environment

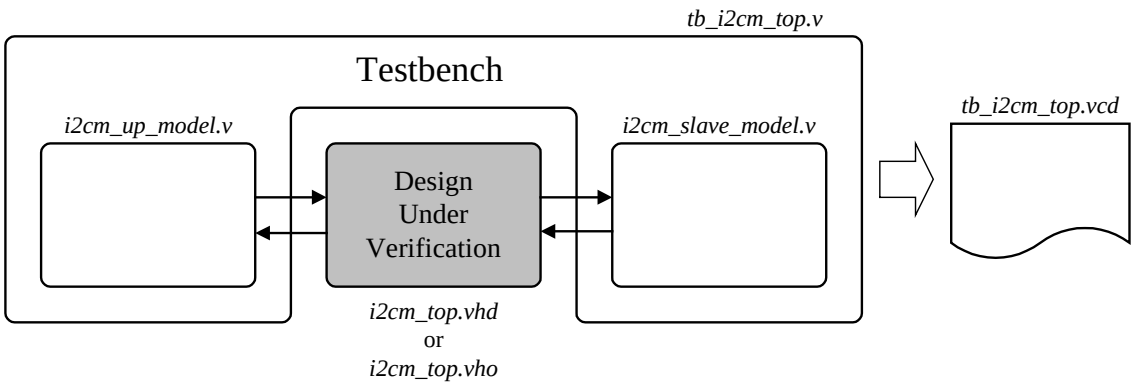
OS	Microsoft Windows 2000 professional
Implementation tool	Altera MAX+plusII 10.1
Synthesis tool	Synplicity Synplify pro 7.0
Simulation tool	Model Technology ModelSim SE 5.5c

2.2. Design flow

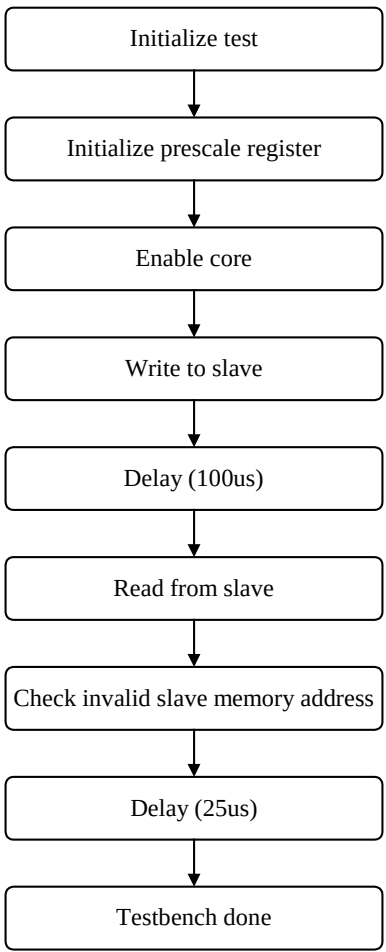


2.3. Testbench

2.3.1. Structure of the testbench and DUV



2.3.2. Flowchart of the testbench (tb_i2cm_top.v)

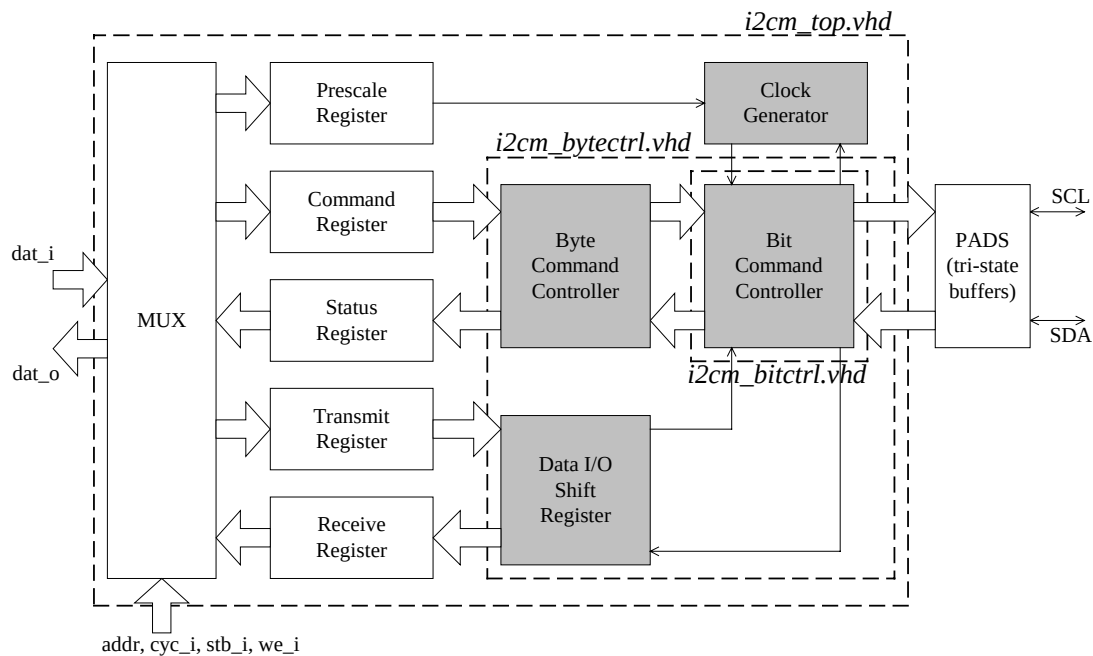


3. Block diagram

The I²C master core is built around four primary blocks, the Clock Generator, the Byte Command Controller, the Bit Command Controller and the Data I/O Shift Register. All other blocks are used for interfacing or for storing temporary values.

The [addr] is a 3-bit bus that distinct each registers, i.e. Prescale Register, Command Register, Transmit Register and Receive Register.

The PADS block must be added at a higher hierarchical level. (FYI: 7.3.1, 9.1)



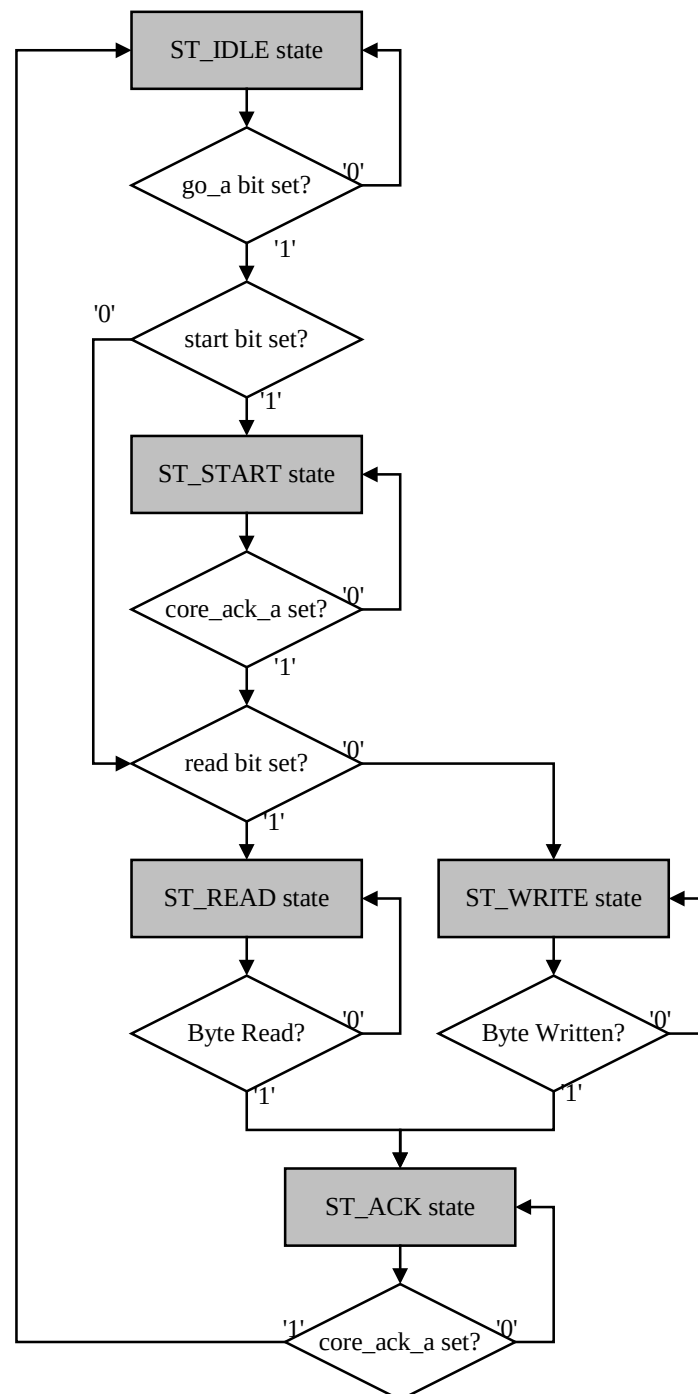
3.1. Clock Generator

The Clock Generator generates an internal 5*SCL clock enable signal that triggers all synchronous elements in the Bit Command Controller. It also handles clock stretching needed by some slaves.

3.2. Byte Command Controller

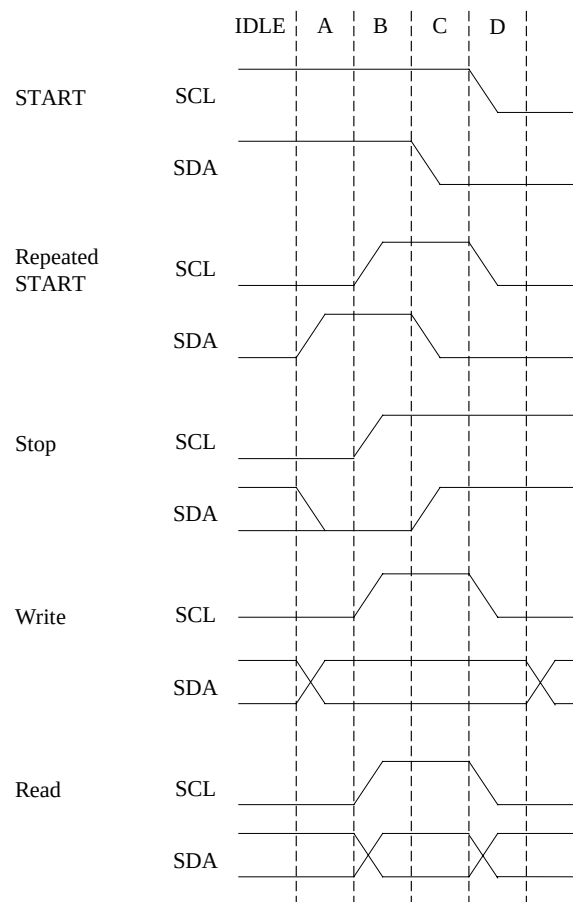
The Byte Command Controller handles I²C traffic at the byte level. It takes data from the Command Register and translates it into sequences based on the transmission of a single byte. By setting the START,

STOP and READ bit in the Command Register, for example, the Byte Command Controller generates a sequence that results in the generation of a START signal, the reading of a byte from the slave devices, and the generation of a STOP signal. It does this by dividing each byte operation into sequence bit-operations, which are then sent to the Bit Command Controller.



3.3. Bit Command Controller

The Bit Command Controller handles the actual transmission of data and the generation of the specific levels for START, Repeated START, and STOP signals by controlling the SCL and SDA lines. The Byte Command Controller tells the Bit Command Controller which operation has to be performed. For a single byte read, the Bit Command Controller receives 8 separate read commands. Each bit-operation is divided into 5 pieces (IDLE and A, B, C, and D), except for a STOP operation which is divided into 4 pieces (IDLE and A, B, and C).



3.4. Data I/O Shift Register

The Data I/O Shift Register contains the data associated with the current transfer. During a read action, data is shifted in from the SDA line. After a byte has been read the contents are copied into the Receive Register. During a write action, the Transmit Register's contents are copied into the Data I/O Shift Register and are then transmitted onto the SDA line.

4. Register description

4.1. Register list

Name	Address	Width	Access	Description
pre_r (low)	0x00	8	R/W	Clock prescale register (low byte)
pre_r (high)	0x01	8	R/W	Clock prescale register (high byte)
ctrl_r	0x02	8	R/W	Control register
rx_r/tx_r	0x03	8	R/W	When access is read, receive register. When access is write, transmit register.
stat_r/cmd_r	0x04	8	R/W	When access is read, status register. When access is write, command register.
tx_r	0x05	8	W	Transmit register (for debugging)
cmd_r	0x06	8	W	Command register (for debugging)
-	0x07	8	R	All zeros

4.2. Register description

4.2.1. Prescale register (name: pre_r)

This register is used to prescale the SCL clock line. Due to the structure of the I2C interface, the core uses a 5*SCL clock internally. The prescale register must be programmed to this 5*SCL frequency (minus 1). Change the value of the prescale register only when the 'EN' bit is cleared.

Example: clk = 32MHz, desired SCL clock = 100KHz

$$prescale = \frac{32MHz}{5 \times 100KHz} - 1 = 63_{(10)} = 3F_{(16)}$$

Reset value: 0xFFFF

4.2.2. Control register (name: ctrl_r)

Bit Number	Access	Description
7	R/W	EN, I2C core enable bit When EN bit set to '1', the core is enabled. When EN bit set to '0', the core is disabled.
6	R/W	IEN, I2C core interrupt enable bit

		When IEN bit set to '1', the core is enabled. When IEN bit set to '0', the core is disabled.
5:0	R/W	<i>Reserved</i>

Reset value: 0x00

The core responds to new commands only when the EN bit is set. Pending commands are finished. Clear the EN bit only when no transfer is in progress, i.e. after a STOP command, or when the command register has the STO bit set. When halted during a transfer, the core can hang the I2C bus.

4.2.3. Transmit register (name: tx_r)

Bit Number	Access	Description
7:1	R/W	Byte, i.e. data or slave address, to transmit on I2C bus
0	R/W	When a data is transferred, this bit is LSB of the data. When a slave address is transferred, this bit is the RW bit. '1' = reading from slave '0' = writing to slave

Reset value: 0x00

4.2.4. Receive register (name: rx_r)

Bit Number	Access	Description
7:0	R	Last byte received on I2C bus

Reset value: 0x00

4.2.5. Command register (name: cmd_r)

Bit Number	Access	Description
7	R/W	STA, generate (repeated) start condition
6	R/W	STO, generate stop condition
5	R/W	RD, read from slave
4	R/W	WR, write to slave
3	R/W	ACK, when a receiver, sent ACK (= '0') or NACK (= '1')
2:1	R/W	<i>Reserved</i>
0	R/W	IACK, Interrupt acknowledge When set, clears a pending interrupt.

Reset value: 0x00

The STA, STO, RD, WR and IACK bits are cleared automatically. These bits are always read as zeros.

4.2.6. Status register (name: stat_r)

Bit Number	Access	Description
7	R	RxACK, Received acknowledge from slave This flag represents acknowledge from the address slave. '1' = No acknowledge received '0' = Acknowledge received
6	R	Busy, I2C bus busy '1' after START signal detected '0' after STOP signal detected
5:2	R	<i>Reserved</i>
1	R	TIP, Transfer in progress '1' when transferring data '0' when transfer complete
0	R	IF, Interrupt flag This bit is set when an interrupt is pending, which will cause an processor interrupt request if the IEN bit is set. The IF is set when one byte transfer has been completed.

Reset value: 0x00

*Note: All **reserved bits** are read as zeros. So they should be written as zeros to ensure forward compatibility.*

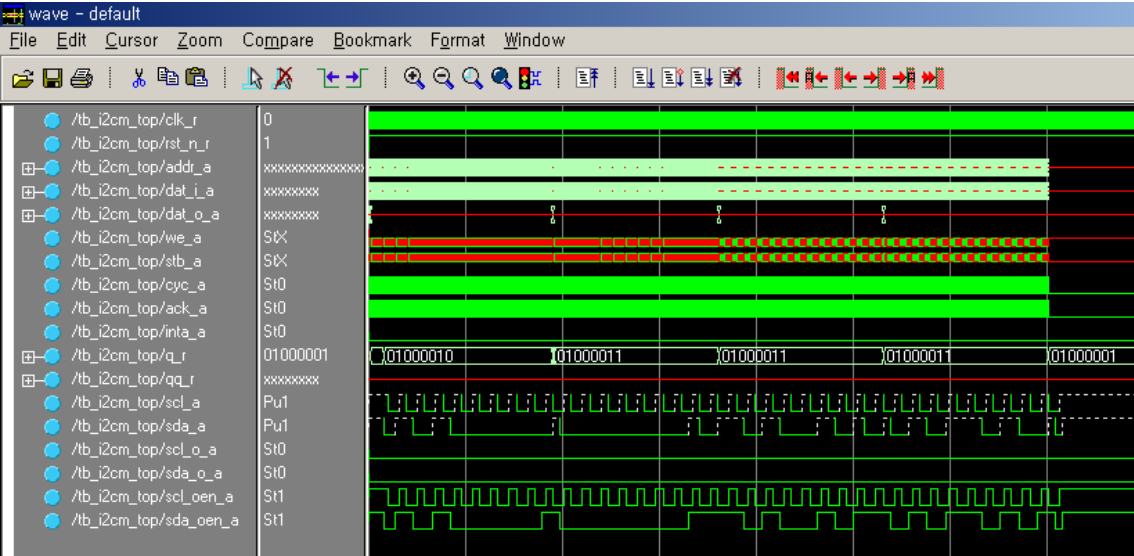
5. Timing diagram

5.1. Simulation result

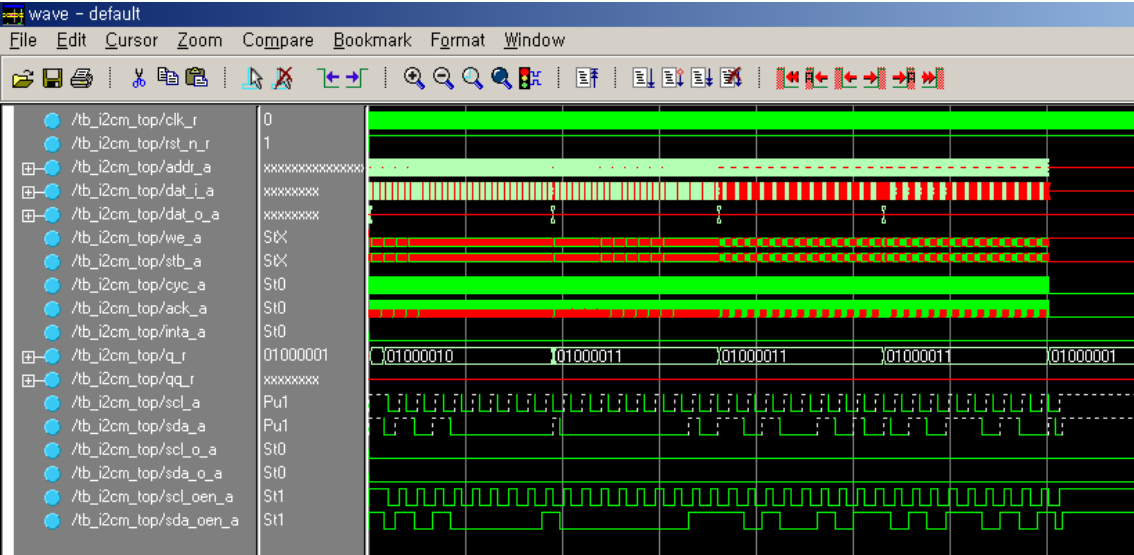
The following figures are that waveforms are simulated by testbench introduced in Section 2.3. The first

one is that DUV is tb_i2cm_top.vhd and the second one is that DUV is tb_i2cm_top.vho. Respectively, the period of [clk] is 15 ns and total simulation time is 500 us.

5.1.1. Functional simulation result



5.1.2. Post layout simulation result



6. Clock distribution

- Active clock edge: rising edge

7. Bus interface & I/O configurations

7.1. Bus interface – I²C protocol

The I²C-bus specification

Version 2.1

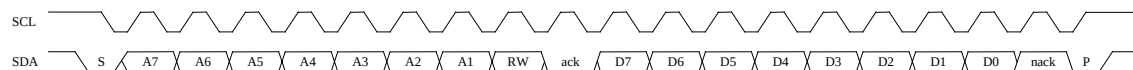
January 2000

Philips Semiconductors

<http://www.semiconductors.philips.com/buses/i2c/>

Normally, a standard communication consists of four parts:

- START signal generation
- Slave address transfer
- Data transfer
- STOP signal generation



7.1.1. START signal

When the bus is free/idle, meaning no master device is engaging the bus (both SCL and SDA lines are high), a master can initiate a transfer by sending a START signal. A START signal, usually referred to as the S-bit, is defined as a high-to-low transition of SDA while SCL is high. The START signal denotes the beginning of a new data transfer. A Repeated START is a START signal without first generating a STOP signal. The master uses this method to communicate with another slave or the same slave in a different transfer direction (e.g. from writing to a device to reading from a device) without releasing the bus.

The core generates a START signal when the STA-bit in the Command Register is set and the RD or WR bits are set. Depending on the current status of the SCL lines, a START or Repeated START is generated.

7.1.2. Slave Address Transfer

The first byte of data transferred by the master immediately after the START signal is the slave address. This is a seven-bits calling address followed by a RW bit. The RW bit signals the slave data transfer direction. No two slaves in the system can have the same address. Only the slave with an address that matches the one transmitted by the master will respond by returning an acknowledge bit by pulling the SDA low at the 9th SCL clock cycle.

Note: the core supports 10 bit slave addresses by generating two address transfers. See the Philips I²C specifications for more details.

The core treats a Slave Address Transfer as any other write action. Store the slave device's address in the Transmit Register and set the WR bit. The core will then transfer the slave address on the bus.

7.1.3. Data Transfer

Once successful slave addressing has been achieved, the data transfer can proceed on a byte-by-byte basis in the direction specified by the RW bit sent by the master. Each transferred byte is followed by an acknowledge bit on the 9th SCL clock cycle. If the slave signals a No Acknowledge, the master can generate a STOP signal to abort the data transfer or generate a Repeated START signal and start a new transfer cycle.

If the master, as the receiving device, does not acknowledge the slave, the slave releases the SDA line for the master to generate a STOP or Repeated START signal.

To write data to a slave, store the data to be transmitted in the Transmit Register and set the WR bit. To read data from a slave, set the RD bit. During a transfer the core set the TIP flags, indicating that a Transfer is In Progress. When the transfer is done, the TIP flag is reset, the IF flag set and, when enable, an interrupt generated. The Receive Register contains valid data after the IF flag has been set. The user may issue write or read command when the TIP flag is reset.

7.1.4. STOP signal

The master can terminate the communication by generating a STOP signal. A STOP signal, usually referred to as the P-bit, is defined as a low-high transition of SDA while SCL is at logical '1'.

7.2. Bus interface – arbitration procedure

Since the I²C core supports single master configurations only, no Arbitration logic is added to the core. Only clock synchronization is supported since slave devices can use this protocol for clock stretching.

7.2.1. Clock Synchronization

Since the logical AND function is performed on the I²C signals, a high to low transition on SCL or SDA affects all devices connected to the bus. The SCL clock signal can be synchronized between multiple masters using this feature. Each device starts counting its SCL low period when the current master drives SCL low. Once a device's clock has gone low, it holds the SCL line low until the clock-high-state is reached.

7.2.2. Clock Stretching

Slave devices can use the clock synchronization mechanism to slow down the transfer bit rate. After the master has driven SCL low, the slave can drive SCL low for the required period and then release it. If the slave's SCL low period is greater than the master's SCL low period, the resulting SCL bus signal low period is stretched, thus inserting wait states.

7.3. I/O list

7.3.1. I/O about I²C bus

Name	Width	Direction	Description
scl_i	1	Input	SCL input
scl_o	1	Output	SCL output
scl_oen	1	Output	SCL output enable
sda_i	1	Input	SDA input
sda_o	1	Output	SDA output
sda_oen	1	Output	SDA output enable

The I²C interface uses a serial data line (SDA) and a serial clock line (SCL) for data transfer. All devices connected to these two signals must have open drain or open collector outputs. Both lines must be pulled-up to VCC by external resistors.

The tri-state buffers for the SCL and SDA lines must be added to at a higher hierarchical level. There is a detailed implementation about the tri-state buffers in the Section 9.

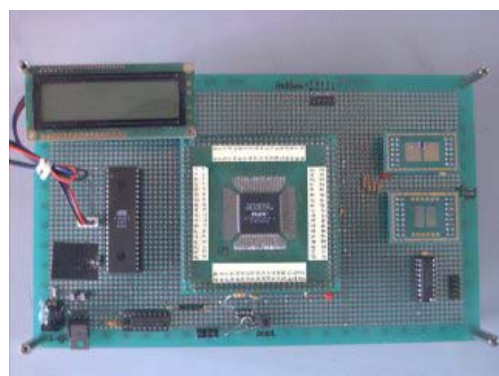
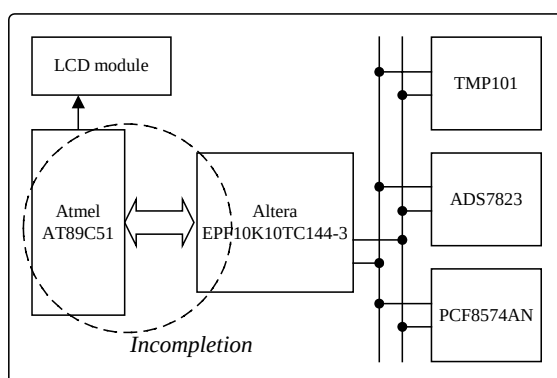
7.3.2. Others I/O

Name	Width	Direction	Description
clk	1	Input	I2C Master controller clock
rst_n	1	Input	Asynchronous reset, active low
rst_sn	1	Input	Synchronous reset, active high
addr	3	Input	Lower address bits
dat_i	8	Input	Data towards the core
stb_i	1	Input	Strobe signal / Core select input
cyc_i	1	Input	Valid bus cycle input
dat_o	8	Output	Data from the core
ack_o	1	Output	Bus cycle acknowledge output
inta_o	1	Output	Interrupt signal output

[rst_n] is provided for FPGA implementations. Using [rst_n] instead of [rst_sn] can result in lower cell-usage and higher performance, because most FPGAs provide a dedicated asynchronous rest path.

8. Test description

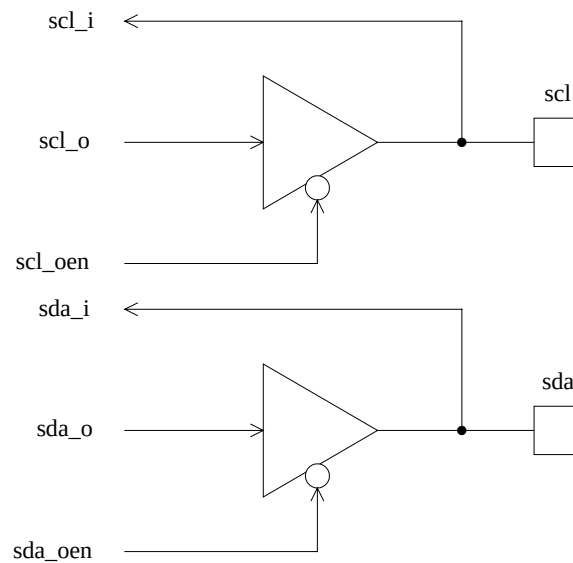
8.1. Test board (progress: 50%)



9. Integration requirements

9.1. External connections

The tri-state buffers for the SCL and SDA lines must be added at a higher hierarchical level. Connections should be made according to following figure:



For FPGA designs the compiler can automatically insert these buffers using that following VHDL code:

```
scl <= scl_o when scl_oen = '0' else 'z' ;  
sda <= sda_o when sda_oen = '0' else 'z' ;  
scl_i <= scl ;  
sda_i <= sda ;
```

Verilog HDL code:

```
assign scl = scl_oen ? 1'bz : scl_o ;  
assign sda = sda_oen ? 1'bz : sda_o ;  
assign scl_i = scl ;  
assign sda_i = sda ;
```

10. Version history

Version	Date	Author	Description
1.0	2002-12-14	Jinil Chung	First draft release

11. Known bugs

None

12. Application note

None

13. Deliverable list / format

File name	Format	Description
i2cm_bitctrl.vhd	VHDL	Bit Command Controller
i2cm_bytectl.vhd	VHDL	Byte Command Controller
i2cm_top.vhd	VHDL	I ² C Master Controller
i2cm_up_model.v	Verilog HDL	Microprocessor Model
i2cm_slave_model.v	Verilog HDL	I2C Slave Model
tb_i2cm_top.v	Verilog HDL	Testbench